

Agentic AI Framework for Risk Scoring, Model Governance, and Monitoring

A Databricks-Enabled LangGraph Multi-Agent Workflow for Risk Scoring, Model Governance, Monitoring, Human-in-the-Loop Review, and Executive Decision Support

Executive Summary



THE PROBLEM

Risk scoring and model governance span multiple disconnected processes, making consistent monitoring, timely issue resolution, and transparent decision support difficult.



THE SOLUTION

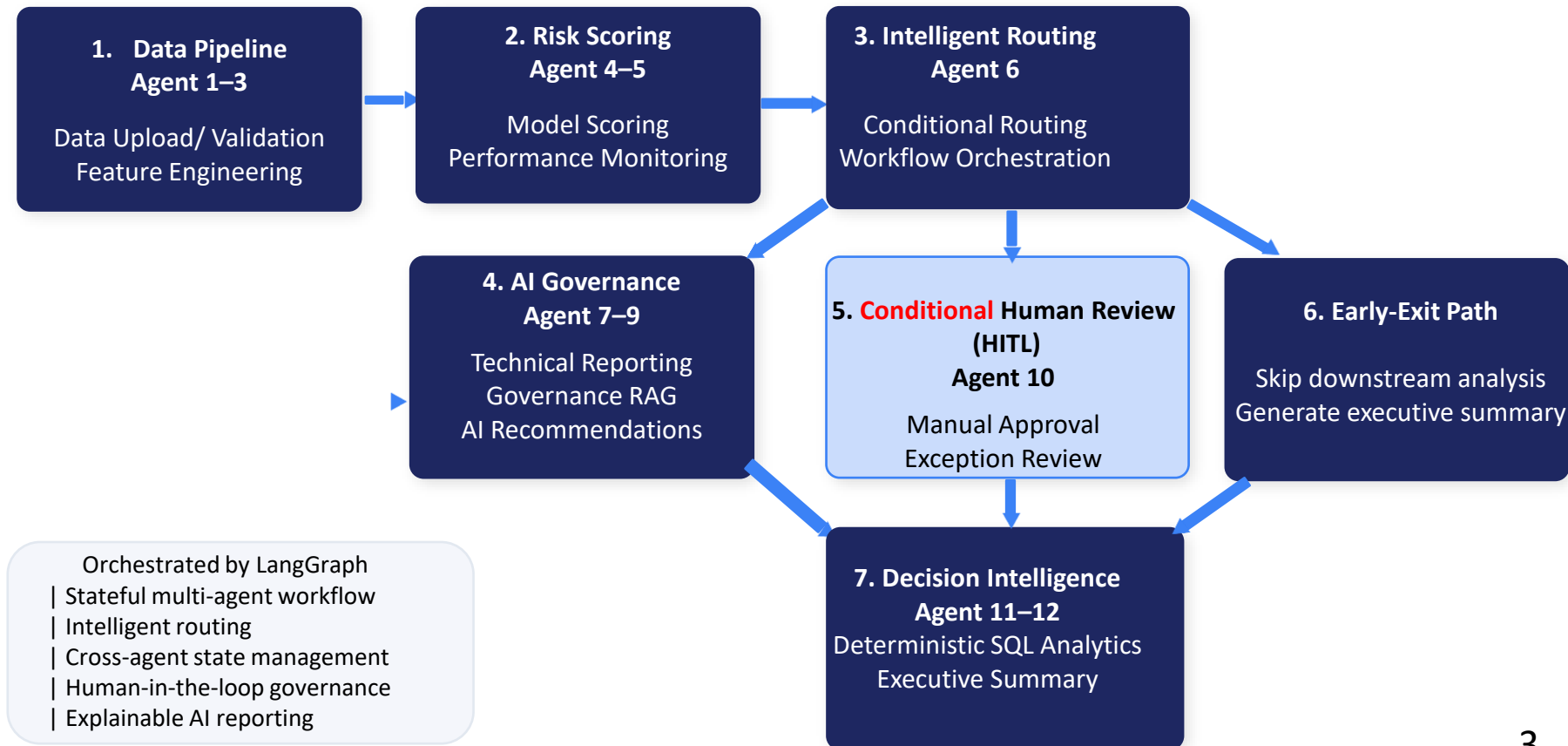
A 12-agent LangGraph risk-scoring and governance workflow migrated from Google Colab to Databricks and engineered with Unity Catalog, Delta Lake, PySpark, modular Python components, failure handling, and end-to-end observability.



THE IMPACT

- 1) Automates end-to-end risk scoring and model governance.
- 2) Integrates ML scoring, governance RAG, HITL review, SQL analytics, and executive reporting.
- 3) Adds engineering controls for reliability, traceability, and failure handling.
- 4) Achieved a Clean Engineering Full Run PASS on Databricks.

LangGraph Agent Architecture



Roadmap & Next Steps

Phase 1 Functional POC — Completed

- ✓ LangGraph multi-agent workflow
- ✓ Automated data validation
- ✓ Feature engineering pipeline
- ✓ ML risk scoring
- ✓ Performance monitoring
- ✓ Conditional workflow routing
- ✓ Governance RAG
- ✓ Human-in-the-loop review
- ✓ Deterministic SQL analytics
- ✓ Executive reporting

Phase 2 Databricks Engineering POC — Completed

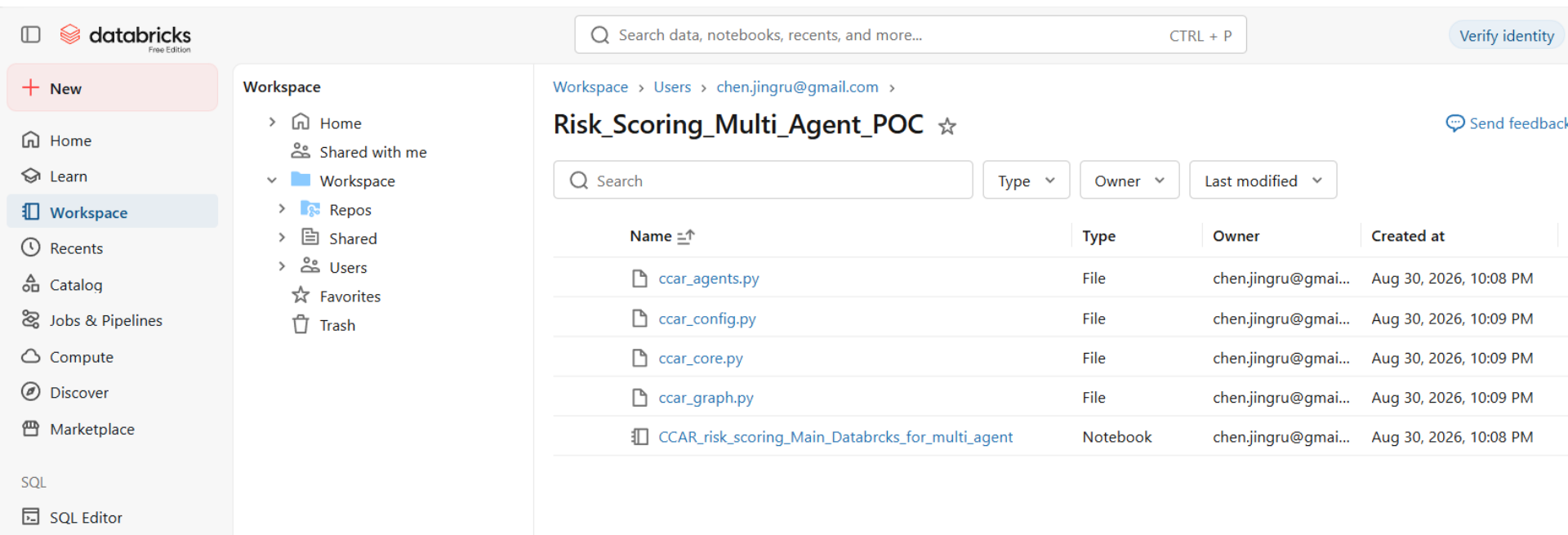
- ✓ Colab → Databricks migration
- ✓ Unity Catalog & Delta Lake
- ✓ PySpark data integration
- ✓ Modular Python architecture
- ✓ Centralized configuration
- ✓ Databricks Secrets
- ✓ Structured logging
- ✓ Exception handling
- ✓ Failure-path validation
- ✓ Token & cost tracking
- ✓ LangSmith observability
- ✓ Clean Engineering Full Run PASS

Phase 3 Productionization — Future

1. CI/CD & automated deployment
2. Enterprise RBAC & security
3. Scalable RAG governance knowledge base
4. Vector database integration
5. Model & data drift monitoring
6. Automated alerts
7. MLOps / LLMOps integration
8. Production audit trail
9. Enterprise approval workflow
10. Production SLA & operational monitoring

Appendix 1: Modular Multi-Agent Risk Scoring Architecture on Databricks

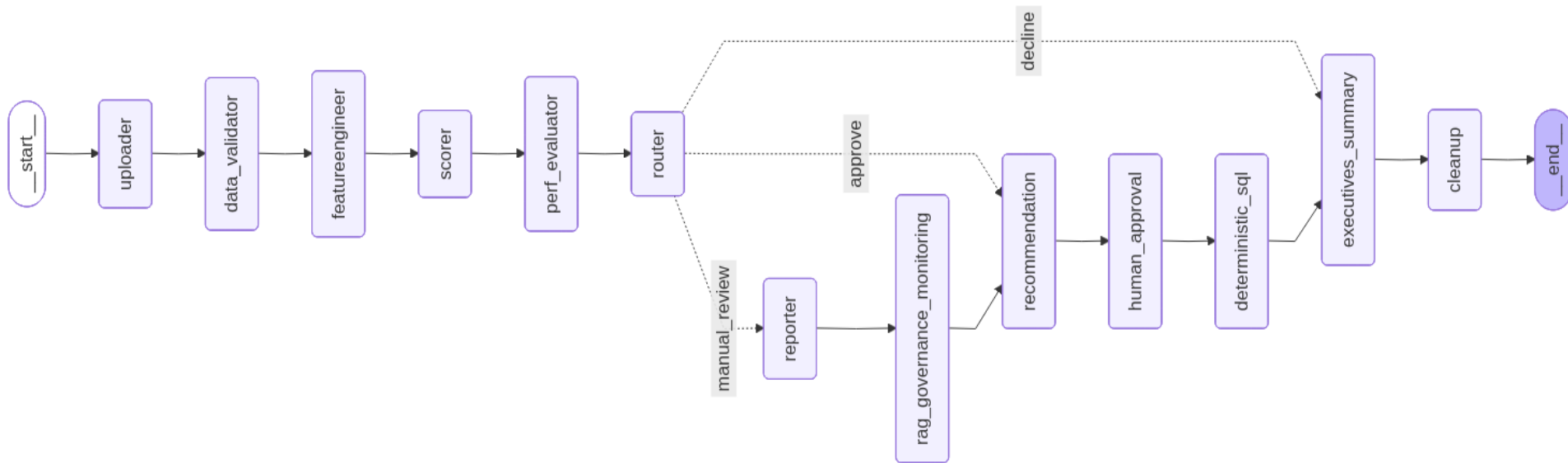
Refactored the original notebook-based prototype into four reusable Python modules and a Databricks orchestration notebook.



The screenshot displays the Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. Below these are SQL and SQL Editor options. The main workspace area shows a breadcrumb path: Workspace > Users > chen.jingru@gmail.com >. The current view is a folder named 'Risk_Scoring_Multi_Agent_POC'. Below the folder name is a search bar and three filter buttons: Type, Owner, and Last modified. A table lists the contents of the folder:

Name	Type	Owner	Created at
ccar_agents.py	File	chen.jingru@gmai...	Aug 30, 2026, 10:08 PM
ccar_config.py	File	chen.jingru@gmai...	Aug 30, 2026, 10:09 PM
ccar_core.py	File	chen.jingru@gmai...	Aug 30, 2026, 10:09 PM
ccar_graph.py	File	chen.jingru@gmai...	Aug 30, 2026, 10:09 PM
CCAR_risk_scoring_Main_Databricks_for_multi_agent	Notebook	chen.jingru@gmai...	Aug 30, 2026, 10:08 PM

Appendix 2: LangGraph Workflow for Risk Scoring & Model Governance



Routing logic:

Manual Review → Technical Report → Governance RAG → Recommendation

Approve → Recommendation → Human Approval

Decline → Executive Summary

Appendix 3: Clean Full Run / Final Outputs

=====

FINAL WORKFLOW OUTPUTS

=====

Technical Report : Generated
RAG Governance : Generated
Recommendation : Generated
SQL Analysis : Generated
Executive Summary : Generated

=====

CCAR DATABRICKS FINAL OUTPUT CHECK

LLM TOKEN & COST SUMMARY

=====

technical_report	: PASS	Model	: gpt-4o-mini
_rag_query	: PASS	Prompt Tokens	: 141
_rag_documents	: PASS	Completion Tokens	: 28
rag_answer	: PASS	Total Tokens	: 169
recommendation_report	: PASS	-----	
approval_status	: PASS	Input Cost	: \$0.00002115
approval_reason	: PASS	Output Cost	: \$0.00001680
sql_query	: PASS	Total LLM Cost	: \$0.00003795
sql_result	: PASS	=====	
executive_summary	: PASS		

=====

Shape: (67957, 45)

validation_report:

	Validation	Status	Details
0	Required columns	PASS	11 required columns found
1	Duplicate Rows	PASS	No duplicates
2	Missing values	WARNING	default_date: 53.1%, default_yrmo: 53.1%
3	YRMO distribution	PASS	49 unique months
4	Target distribution	PASS	Bad rate = 2.50%
5	Outliers	WARNING	5227 values outside 1%-99% range

scoring_report:

	Vintage	Status	Count	Avg_PD	Precision	Recall	F1	TP	FN	TN	FP
0	201604	Success	1179	0.332076	0.033058	0.181818	0.055944	4	18	1040	117

performance_report:

	Metric	Status	Value	Details
0	AUC	WARNING	0.6388	
1	KS	PASS	0.2517	
2	Precision	INFO	0.0331	
3	Recall	INFO	0.1818	
4	F1	INFO	0.0559	
5	Actual Bad Rate	INFO	0.0187	
6	Predicted Bad Rate	INFO	0.1026	
7	Average PD	INFO	0.3321	

Appendix 4: End-to-End Multi-Agent Observability with LangSmith

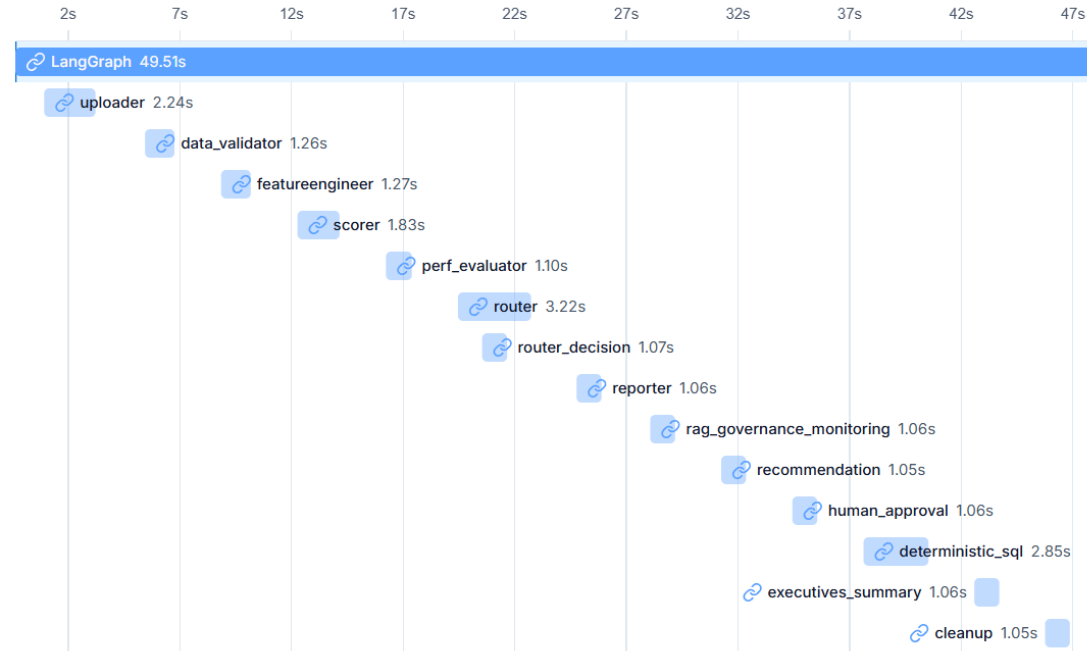
<https://smith.langchain.com/o/ad5597f9-6d53-4729-ae1f-c92aa4a2b5b1/projects/p/bb1632dd-b342-476c-8697-37ca72095b73/trace/01a0!>

Personal / Tracing / JCHEN-Databricks-credit-risk_s... / LangGraph

<< LangGraph ID

Trace

Waterfall Reset



LangSmith Runtime Observability

- End-to-end LangGraph workflow successfully traced
- Node-level execution path and latency captured
- Full workflow runtime: **49.51 seconds**

Appendix 5: Migration from Google Colab to Databricks

Phase 1 — Functional POC	Phase 2 — Databricks Engineering POC
Google Colab-based prototype	Colab → Databricks migration
End-to-end 12-agent workflow	Modular Python architecture
ML risk scoring	Unity Catalog + Delta Lake
Governance RAG	PySpark data integration
Human-in-the-loop approval	Centralized configuration
Deterministic SQL analytics	Databricks Secrets
Executive reporting	Structured logging + exception handling
Functional Full Run PASS	Failure-path testing
	Token & cost tracking
	LangSmith observability
	Clean Engineering Full Run PASS

Thank You

Questions & Discussion
